AI-Powered Text Intelligence Platform

Sentiment Analysis of Amazon Food Reviews: From Data to Deployment

CSE 3521: Survey of Artificial Intelligence I | Autumn 2026

Type:	Group Project	**Teams:**	4–6 students
Dataset:	Amazon Fine Food Reviews	**Total:**	100 points
Platform:	Streamlit (deployed)	**Final demo:**	See Carmen
Window:	Nov – Dec 2026	**Repo:**	GitHub (public)

Background & Motivation

Online reviews are one of the richest sources of real-world natural language data. Millions of consumers read and write reviews every day, and the ability to automatically understand their sentiment — whether a review is positive, negative, or neutral — is a core capability in search, recommendation, content moderation, and brand analytics.

This project asks your team to build and deploy a **text intelligence system** for the Amazon Fine Food Reviews dataset using the machine learning methods taught in this course. Rather than following a prescribed recipe, your team makes the key design decisions: which features to engineer, which models to build and compare, and how to present your findings to a non-technical audience through a live application.

The goal is not just working code. It is the ability to **scope a problem, plan an approach, execute iteratively, and communicate results clearly** — the same cycle used in industry AI teams.

Dataset

Amazon Fine Food Reviews
Source: https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews
Size: 568,454 reviews with text, star ratings (1–5), timestamps, and helpfulness votes.

Your team will work from a shared 50,000-review sample to keep computation manageable. The primary inputs are the **review text** and **summary** fields. The primary target is the **star rating**, which you will transform into a sentiment label of your choosing (e.g., positive / negative, or a three-class scheme).

All teams work on the same dataset. Your differentiation comes from your modeling choices, evaluation depth, and presentation quality.

Technical Scope

Your system must include **all three layers** below:

Layer	Requirement
Feature Engineering	Convert raw review text into numerical representations suitable for machine learning. You must implement at least two distinct feature representations (e.g., Bag of Words, TF-IDF, word embeddings, character N-grams). Justify your choices.
Classical Models	Train and evaluate at least two classical ML models (e.g., Naïve Bayes, Logistic Regression, Gaussian Mixture Model, k-Nearest Neighbors). Tune hyperparameters and compare performance.
Deep / Modern Model	Train and evaluate at least one deep learning or modern NLP model (e.g., Feedforward Neural Network, LSTM, or a pre-trained transformer such as DistilBERT). Compare it to your classical baselines.
Deployed Application	Package your pipeline into a Streamlit web application deployed to Streamlit Community Cloud. The app must accept new review text and return live predictions from at least two of your trained models.

Evaluation metrics. Report at minimum: accuracy, precision, recall, and F1-score. Include a confusion matrix for each model. If you frame the task as regression (predicting the raw star score), report RMSE and MAE instead.

Team Structure & Roles

Teams of **4–6 students** will be formed by the instructor. Every team member must have a named primary role, documented in Milestone 1. Suggested roles are below; adapt them to your team size.

Role	Primary responsibilities
Project Lead	Coordinates weekly check-ins, owns the milestone timeline, resolves blockers, integrates everyone's work.
Data Engineer	Loads and cleans the dataset, builds the feature engineering pipeline, maintains shared data splits.
ML Engineer(s)	Implements, trains, and evaluates the classical and deep models; owns the model comparison table.
App Engineer	Builds and deploys the Streamlit application; ensures the live prediction pipeline works end-to-end.
Analyst / Writer	Writes the final report, synthesizes results, prepares the presentation deck.

In teams of 4, roles may be combined (e.g., one person covers both Data Engineer and ML Engineer). In teams of 6, the ML Engineer role may be split between classical models and the deep model. Every member is expected to contribute code or analysis in addition to their primary role.

Milestones & Deliverables

The project runs across four milestones over four weeks. Submissions are made through Carmen unless noted otherwise. **Late submissions: 10% per day; no credit after 3 days.**

Milestone 1 | Project Charter & Scope — Due: Sun Nov 8, 11:59 pm

Purpose. Before writing a single line of code, your team aligns on the problem definition, divides responsibilities, and commits to a realistic plan. This mirrors the *project kick-off* and *scoping document* used in industry AI teams.

Required sections. Submit a single PDF (2–3 pages, shared GitHub link at the top):

1. **Team roster.** List each member's name, OSU email, and primary role. Include a one-sentence statement from each member describing what they will own.

2. **Problem statement.** In 1–2 paragraphs, describe the business or research question your system will answer. Who would use this system? What decision does the prediction support?
3. **Dataset summary.** Report: number of rows, class distribution of your chosen sentiment labels, average review length, and any data quality issues you observed (missing values, class imbalance, duplicates). Include one visualization (e.g., score distribution bar chart).
4. **Technical approach.** Describe which feature representations and models you plan to build and why. Explicitly connect each modeling choice to a course lesson (e.g., "We will implement Naive Bayes from Lesson 9 as our first classifier").
5. **Project timeline.** A week-by-week table showing who is doing what between now and Dec 2. Identify at least one risk and your mitigation plan.
6. **GitHub repository.** Create a public repository with a populated `README.md` (project title, team members, brief description). Include the link in your document.

Deliverable: PDF via Carmen + public GitHub repository URL *2–3 pages*

[15 pts] Rubric: Team alignment and role clarity (4); Problem framing and dataset insight (5); Technical plan connected to course content (4); Timeline realism and risk identification (2).

Milestone 2 | Implementation Plan & Initial Solution **Due: Sun Nov 15, 11:59 pm**

Purpose. Your team has explored the data, built your feature pipeline, and trained first versions of at least one classical model. This mirrors an industry *proof-of-concept* review: something that runs end-to-end, even if not yet polished.

Required sections. Submit a PDF (3–5 pages) plus a link to your updated GitHub repository:

1. **Feature engineering report.** Describe the text representations you built. For each one: explain the method, report the vocabulary size or embedding dimension, and show the top-10 most informative features for predicting positive and negative sentiment. Include at least one visualization (e.g., word frequency chart, PCA scatter of your feature space).
2. **Baseline results.** Report accuracy, precision, recall, and F1-score on a held-out test set (20% of data) for at least one trained classical model. Include a confusion matrix. Compare to a majority-class baseline (always predict the most common label) to establish whether your model is learning anything.
3. **Error analysis.** Examine 5–10 misclassified examples. Identify at least one pattern (e.g., short reviews, sarcasm, reviews with numeric emphasis) that your current model handles poorly. How might you address this?

4. **Updated timeline.** Revise your Milestone 1 timeline based on what you have learned. Note any scope changes and their rationale.
5. **Individual progress log.** One paragraph per team member: what they completed since M1, what they are working on next.

Deliverable: PDF via Carmen + updated GitHub repository *3–5 pages*

[20 pts] Rubric: Feature engineering quality and depth (6); Baseline model correctness and evaluation completeness (7); Error analysis insight (5); Progress log accountability (2).

Milestone 3 | Refined Solution & App Prototype

Due: Sun Nov 22, 11:59 pm

Purpose. All required models are trained and compared. The Streamlit app is running locally or in a staging deployment. This mirrors an industry *internal demo*: a working product presented to stakeholders before final polish.

Required sections. Submit a PDF (4–6 pages) plus a link to your GitHub repository with the app code:

1. **Model comparison table.** Present a unified table with one row per trained model and columns for accuracy, precision, recall, F1-score, and training time. Include both classical models and your deep/modern model. Identify the best-performing model and explain why it outperforms the others in terms of the course concepts covered (e.g., capacity, feature representation, handling of sequence information).
2. **Deep / modern model analysis.** Describe the architecture you chose and why. Report training and validation loss curves. Discuss one advantage this model has over your classical baselines and one limitation (cost, data requirement, interpretability, etc.).
3. **Streamlit app description.** Document the tabs or pages in your app, what each one shows, and how a user would interact with it. Include at least two screenshots of the running app. Provide a local run command (`streamlit run app.py`) and note the deployment target URL if already live.
4. **Reproducibility note.** Describe how another team could reproduce your results: where to download the data, how to install dependencies, how to retrain the models. This should be captured in your `README.md`.
5. **Individual progress log.** One paragraph per team member: contribution since M2 and remaining tasks.

Deliverable: PDF via Carmen + GitHub with app code + Streamlit URL (if deployed) *4–6 pages*

[30 pts] Rubric: Model comparison completeness and correctness (10); Deep model analysis depth (8); App functionality and documentation (8); Reproducibility (2); Progress log accountability (2).

Milestone 4 | Final Report & Live Presentation **Due: Sun Nov 29 (report) / See Carmen (demo)**

Purpose. The finished system is documented and presented to the class. This mirrors an industry *project close-out*: a written report that could onboard a future engineer, and an executive-style demo that communicates results to a non-technical audience.

4a. Final Report [25 pts]

Submit a PDF (8–12 pages, excluding appendices) structured as follows. The report should read as a coherent document, not a collection of bullet points.

1. **Executive summary** (0.5 page). What problem did you solve? What is your best result? Who would benefit from this system?
2. **Dataset and feature engineering** (1–2 pages). Summarize the dataset and the feature representations you built. Justify your final choices.
3. **Models and results** (2–3 pages). Present the full model comparison table. Discuss the most important findings: which model works best and why, where all models struggle, and what the error analysis revealed about the limits of your approach.
4. **System description** (1 page). Describe the deployed application: what it does, how it is structured, and how a user would interact with it. Include the live URL.
5. **Individual contributions** (0.5–1 page). One short paragraph per member describing their specific contribution to the project. Be concrete (e.g., "Built the TF-IDF pipeline and trained the Logistic Regression model; wrote Sections 2 and 3 of the report").
6. **Lessons learned and future work** (0.5–1 page). What would you do differently? What would you add with more time? What surprised you about working with real text data?
7. **References.** Cite any papers, documentation, or resources used.
8. **Appendix (optional).** Additional figures, ablation results, or supplementary tables that did not fit in the main body.

Formatting: 12pt font, 1-inch margins, single-spaced. Figures and tables are numbered and captioned. Section headings are clear. Proofread before submission.

Deliverable: PDF via Carmen by Sun Nov 29 at 11:59 pm

4b. Live Demo Presentation [10 pts]

Format	8-minute presentation + 2 minutes Q&A
Audience	Class + instructor
Date	See Carmen
Slides	Submit to Carmen by Sun Nov 29 at 11:59 pm

Required demo sequence. Your presentation must include, in order:

1. A one-sentence statement of the problem and why it matters.
2. A brief data overview (30 seconds, one chart).
3. A live demonstration: type or paste a review into your Streamlit app and show the predictions from at least two models side-by-side.
4. The model comparison table with your headline finding.
5. One thing that did not work as expected and what you learned from it.

Every team member must speak during the presentation. The Q&A may include questions directed at specific members about their contributions. Grading criteria are detailed in the course syllabus on Carmen.

Point Summary

Milestone	Due	Points
M1: Project Charter & Scope	Sun Nov 8	15
M2: Implementation Plan & Initial Solution	Sun Nov 15	20
M3: Refined Solution & App Prototype	Sun Nov 22	30
M4a: Final Report	Sun Nov 29	25
M4b: Live Demo Presentation	See Carmen	10
Total		**100**

GitHub Repository Requirements

Your public GitHub repository must be set up by M1 and maintained throughout the project. The repository structure should follow this convention:

File / Folder	Contents
`README.md`	Project title, team, live app URL, setup instructions
`data/`	Instructions for downloading the dataset (do not commit the CSV)
`notebooks/`	Jupyter notebooks for EDA, feature engineering, model training
`models/`	Saved model weights or serialized model files
`app.py`	Streamlit application entry point
`requirements.txt`	All Python dependencies with version pins
`report/`	Final report PDF and presentation slides

Commit history matters. A repository with one large final commit raises academic integrity concerns. Commit regularly so your individual contributions are visible.

Grading Philosophy

This project is graded on **process and communication** as much as on technical performance. A team whose best model reaches 85% accuracy but writes a clear, honest report — including what did not work — will score higher than a team that claims 92% accuracy in a poorly written report that cannot be reproduced.

Your models do not need to be state-of-the-art. They need to be correctly implemented, honestly evaluated, and clearly explained.

Academic Integrity

- Each team submits one shared deliverable per milestone.
- You may use any Python library for implementation. If a course lesson requires implementation from scratch (e.g., gradient descent in HW3), you may still use a library for the project — but you must understand what it is doing and be able to explain it during Q&A.
- You may consult documentation, tutorials, and papers. You may not copy another team's code or report.
- AI coding assistants (GitHub Copilot, ChatGPT, etc.) may be used for implementation assistance. You must disclose usage in your final report (one sentence is sufficient). All analysis, interpretation, and writing must be your own.
- Every member must be able to explain any part of the project during Q&A. If a member cannot explain their team's work, their individual score may be adjusted.

Support

Post all questions on the **Carmen discussion board** (tag: `#project`). For technical blockers (environment setup, Streamlit deployment, Kaggle access), post early — most issues take 24 hours to resolve with help, and much longer when discovered the night before a deadline.

One piece of advice: The teams that do well treat the first milestone as genuinely important, not as a box to check. The decisions you make in Week 1 — about scope, about who owns what, about what "done" looks like — determine how the rest of the month goes. Start early. Communicate often. Document as you go.